

Answer 1: Linear Regression with a Binary Feature

(a) Least-squares estimators

The residual sum of squares is

$$\text{RSS}(\alpha, \gamma) = \sum_{i=1}^n [y_i - \alpha x_i - \gamma(1 - x_i)]^2.$$

Because $x_i \in \{0, 1\}$, the predicted response is α when $x_i = 1$ and γ when $x_i = 0$. Therefore,

$$\text{RSS}(\alpha, \gamma) = \sum_{i:x_i=1} (y_i - \alpha)^2 + \sum_{i:x_i=0} (y_i - \gamma)^2.$$

Taking partial derivatives and setting them equal to zero,

$$\frac{\partial \text{RSS}}{\partial \alpha} = -2 \sum_{i:x_i=1} (y_i - \alpha) = 0,$$

$$\frac{\partial \text{RSS}}{\partial \gamma} = -2 \sum_{i:x_i=0} (y_i - \gamma) = 0.$$

Solving these equations gives

$$\hat{\alpha} = \frac{1}{n_1} \sum_{i:x_i=1} y_i = \frac{\sum_{i=1}^n x_i y_i}{n_1}$$

and

$$\hat{\gamma} = \frac{1}{n_0} \sum_{i:x_i=0} y_i = \frac{\sum_{i=1}^n (1 - x_i) y_i}{n_0}.$$

Each term in the separated RSS is a strictly convex quadratic in its corresponding coefficient, since

$$\frac{\partial^2 \text{RSS}}{\partial \alpha^2} = 2n_1 > 0, \quad \frac{\partial^2 \text{RSS}}{\partial \gamma^2} = 2n_0 > 0.$$

Because the objective separates into these two terms, the estimates give the unique minimum.

Interpretation: $\hat{\alpha}$ is the sample mean of the responses in the $X = 1$ group, while $\hat{\gamma}$ is the sample mean of the responses in the $X = 0$ group.

(b) Why the estimators are random variables

Although the x_i values are fixed, each response y_i depends on the random error ϵ_i . Substituting the model into the estimators gives

$$\hat{\alpha} = \alpha + \frac{1}{n_1} \sum_{i:x_i=1} \epsilon_i, \quad \hat{\gamma} = \gamma + \frac{1}{n_0} \sum_{i:x_i=0} \epsilon_i.$$

Therefore, both estimators are functions of random errors and vary across repeated realizations of the data.

(c) Expected values and variances

Using $\mathbb{E}[\epsilon_i] = 0$ and treating the group memberships and group sizes as fixed,

$$\begin{aligned}\mathbb{E}[\hat{\alpha}] &= \alpha + \frac{1}{n_1} \sum_{i:x_i=1} \mathbb{E}[\epsilon_i] = \alpha, \\ \mathbb{E}[\hat{\gamma}] &= \gamma + \frac{1}{n_0} \sum_{i:x_i=0} \mathbb{E}[\epsilon_i] = \gamma.\end{aligned}$$

Hence,

$$\boxed{\mathbb{E}[\hat{\alpha}] = \alpha, \quad \mathbb{E}[\hat{\gamma}] = \gamma}.$$

Both estimators are unbiased.

Because the errors are independent, the variance of each sum is the sum of the variances. Using $\text{Var}(\epsilon_i) = \sigma^2$,

$$\begin{aligned}\text{Var}(\hat{\alpha}) &= \frac{1}{n_1^2} \text{Var}\left(\sum_{i:x_i=1} \epsilon_i\right) \\ &= \frac{1}{n_1^2} \sum_{i:x_i=1} \text{Var}(\epsilon_i) = \frac{n_1 \sigma^2}{n_1^2} = \frac{\sigma^2}{n_1}, \\ \text{Var}(\hat{\gamma}) &= \frac{1}{n_0^2} \text{Var}\left(\sum_{i:x_i=0} \epsilon_i\right) \\ &= \frac{1}{n_0^2} \sum_{i:x_i=0} \text{Var}(\epsilon_i) = \frac{n_0 \sigma^2}{n_0^2} = \frac{\sigma^2}{n_0}.\end{aligned}$$

Thus,

$$\boxed{\text{Var}(\hat{\alpha}) = \frac{\sigma^2}{n_1}, \quad \text{Var}(\hat{\gamma}) = \frac{\sigma^2}{n_0}}.$$

A larger group size produces a smaller variance for its corresponding estimator.

(d) Independence of the estimators

Yes, $\hat{\alpha}$ and $\hat{\gamma}$ are statistically independent.

The estimator $\hat{\alpha}$ depends only on the errors $\{\epsilon_i : x_i = 1\}$, while $\hat{\gamma}$ depends only on the errors $\{\epsilon_i : x_i = 0\}$. These are disjoint collections of independent random variables.

Functions of these independent collections are themselves independent. Therefore,

$$\boxed{\hat{\alpha} \text{ and } \hat{\gamma} \text{ are independent.}}$$

This conclusion does not require normally distributed errors.

(e) Comparison of the two models

Model (2) does not offer greater flexibility than model (1). Their prediction functions are

$$f_1(x) = \alpha x + \gamma(1 - x), \quad f_2(x) = \beta_0 + \beta_1 x.$$

Rearranging the first prediction function,

$$f_1(x) = \gamma + (\alpha - \gamma)x.$$

Thus, the functions are identical under the correspondence

$$\boxed{\beta_0 = \gamma, \quad \beta_1 = \alpha - \gamma}.$$

Conversely,

$$\boxed{\gamma = \beta_0, \quad \alpha = \beta_0 + \beta_1}.$$

For the two possible feature values,

	$x = 0$	$x = 1$
$f_1(x)$	γ	α
$f_2(x)$	β_0	$\beta_0 + \beta_1$

Both models can assign any real-valued prediction to each of the two groups. They therefore describe exactly the same set of prediction functions and have equal flexibility.

In model (2), β_0 represents the prediction for the $X = 0$ group, and β_1 represents the difference between the predictions for the $X = 1$ and $X = 0$ groups.

Answer 2: Predicting Honda Accord Sales

Data preparation

The data were sorted chronologically and divided into:

- Training set: August 2016–July 2023, containing 84 observations.
- Testing set: August 2023–July 2026, containing 35 observations.

All model fitting, variable selection, and evaluation reported below use only the training set. The testing set is reserved for later evaluation.

All numerical predictors retain their original units. An intercept is included in every model.

(a) Selecting a regression model

(i) Regression equation and interpretation

The selected model includes `AccordQueries`, `CPIEnergy`, and `MilesTraveled`.

Let Q denote normalized Accord search interest, E denote energy CPI, and M denote vehicle miles traveled, measured in millions of miles. The fitted prediction equation is

$$\widehat{\text{AccordSales}} = 4072.87 + 166.63Q - 118.78E + 0.122416M.$$

The predicted response is the number of Accords sold in the United States during a month.

Holding the other included predictors constant:

- **AccordQueries:** A one-point increase in normalized search interest is associated with approximately 166.63 additional Accord sales per month.
- **CPIEnergy:** A one-index-point increase in energy CPI is associated with approximately 118.78 fewer Accord sales per month. An index-point increase is not the same as a one-percent increase.
- **MilesTraveled:** An additional one million vehicle miles traveled is associated with approximately 0.1224 additional Accord sales per month. Equivalently, an additional one billion miles is associated with approximately 122.42 additional sales. The predictor remains in its original units in the fitted model.
- **Intercept:** The model predicts 4072.87 sales when all three predictors equal zero. This combination is outside the relevant observed range, so the intercept has limited practical interpretation.

These coefficients describe conditional associations, not necessarily causal effects.

(ii) Variable-selection procedure

I evaluated all

$$2^5 - 1 = 31$$

nonempty subsets of the five permitted predictors: `Unemployment`, `AccordQueries`, `CPIEnergy`, `CPIAll`, and `MilesTraveled`.

To preserve chronological order, I used five expanding-window validation folds entirely within the training set:

Fold	Fitting observations	Validation observations
1	1–24	25–36
2	1–36	37–48
3	1–48	49–60
4	1–60	61–72
5	1–72	73–84

For each subset, I fitted the model on each fitting window and predicted the following 12 observations. I pooled the 60 validation errors and calculated

$$\text{RMSE}_{CV} = \sqrt{\frac{1}{60} \sum_{i \in \mathcal{V}} (y_i - \hat{y}_i^{\text{fold}})^2},$$

where $\mathcal{V}$ contains the validation observations.

The subset with the lowest validation RMSE was selected and refitted using all 84 training observations.

Predictor subset	Validation RMSE
AccordQueries, CPIEnergy, MilesTraveled	4732.20
CPIEnergy, MilesTraveled	4733.40
CPIAll, MilesTraveled	4893.59
AccordQueries, CPIAll, MilesTraveled	4953.51

The selected three-predictor model only narrowly outperformed the two-predictor model. Therefore, there is little evidence of a meaningful predictive advantage from including `AccordQueries`, although the minimum-RMSE selection rule retains it.

Because these validation results were used for model selection, they are not an independent final assessment of future performance.

(iii) Coefficient signs and uncertainty

The coefficient signs have plausible interpretations:

- The positive search-interest coefficient is consistent with greater consumer interest being associated with higher sales.

- The negative energy-CPI coefficient is consistent with higher vehicle operating costs discouraging purchases. However, substitution toward fuel-efficient vehicles could work in the opposite direction, so this sign is not economically inevitable.
- The positive mileage coefficient is consistent with greater driving activity being associated with greater vehicle demand.

The conventional OLS results are:

Predictor	Estimate	<i>p</i> -value	95% confidence interval
AccordQueries	166.6341	0.0392	[8.44, 324.82]
CPIEnergy	−118.7804	< 0.0001	[−141.64, −95.92]
MilesTraveled	0.122416	< 0.0001	[0.07614, 0.16870]

All three intervals exclude zero, providing nominal statistical support for the estimated signs. Evidence is stronger for energy CPI and mileage than for search interest.

The conventional *t*-tests and intervals assume a correctly specified linear model with independent, constant-variance errors; normal errors justify their exact finite-sample distributions. Monthly errors may violate these assumptions, and these intervals do not account for variable selection. Therefore, the results support the signs within this fitted model but do not guarantee stable signs in future periods.

(iv) Training performance

For training responses y_i and fitted values $\hat{y}_i$,

$$R^2 = 1 - \frac{\sum_{i=1}^{84} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{84} (y_i - \bar{y})^2},$$

and

$$\text{RMSE} = \sqrt{\frac{1}{84} \sum_{i=1}^{84} (y_i - \hat{y}_i)^2}.$$

The selected model gives

$$\boxed{R^2 = 0.6387, \quad R_{\text{adj}}^2 = 0.6251, \quad \text{RMSE} = 3663.45.}$$

Thus, it explains approximately 63.87% of the variation in training sales. Its training RMSE is approximately 3663 cars per month.

For comparison, predicting the training mean for every observation gives an RMSE of 6094.49 cars. The regression therefore improves substantially over this baseline, although considerable unexplained variation remains. Training performance alone does not establish future predictive accuracy.

(v) Comparison with all five predictors

Measure	Selected model	All five predictors
Number of predictors	3	5
Training R^2	0.6387	0.7092
Adjusted R^2	0.6251	0.6906
Training RMSE	3663.45	3286.47
Chronological validation RMSE	4732.20	15139.30

The full model fits the training data better, including after the adjusted- R^2 penalty. However, it performs much worse in the specified chronological validation procedure. This supports choosing the smaller model for prediction under this validation design.

The validation result depends on the chosen historical windows, which begin with only 24 fitting observations. It does not guarantee that the full model will perform worse in every future period.

The full model's conventional coefficient p -values are:

Predictor	p -value
Unemployment	0.0051
AccordQueries	0.1739
CPIEnergy	0.0064
CPIAll	0.1648
MilesTraveled	0.000145

At the 5% level, unemployment, energy CPI, and mileage are significant in the full model. Search interest and overall CPI are not. In the selected model, all three included predictors are nominally significant.

To assess multicollinearity, I calculated

$$\text{VIF}_j = \frac{1}{1 - R_j^2},$$

where R_j^2 comes from regressing predictor j on the remaining predictors with an intercept.

Predictor	Selected-model VIF	Full-model VIF
Unemployment	—	2.18
AccordQueries	1.56	1.75
CPIEnergy	1.07	10.90
CPIAll	—	8.99
MilesTraveled	1.54	1.84

Energy CPI and overall CPI have a training correlation of approximately 0.907. Their large VIFs in the full model indicate substantial multicollinearity, making their individual effects harder to distinguish and potentially less stable.

The selected model has low VIFs for all included predictors. Multicollinearity is a plausible contributor to coefficient instability, although it alone does not establish the cause of the full model's larger validation error.

(b) Modeling seasonality

(i) Regression equation and month effects

I fitted a model with all five numerical predictors and **MonthFactor**, without variable selection. January is the reference month.

Let U, Q, E, C, M denote unemployment, search interest, energy CPI, overall CPI, and mileage, respectively. Define

$$D_m = \begin{cases} 1, & \text{if the observation is in month } m, \\ 0, & \text{otherwise.} \end{cases}$$

The fitted model is

$$\widehat{\text{AccordSales}} = 29883.43 - 1122.05U + 272.69Q \\ - 125.77E - 27.53C + 0.036875M + \sum_{m=2}^{12} \hat{\delta}_m D_m.$$

The estimated month coefficients are:

Month	$\hat{\delta}_m$	p -value
January (reference)	0	—
February	1839.86	0.3469
March	3384.02	0.0870
April	4802.53	0.0176
May	6709.92	0.0127
June	3637.74	0.1614
July	1457.55	0.6041
August	4267.05	0.1154
September	4074.09	0.0745
October	3552.99	0.1827
November	4508.02	0.0237
December	7088.93	0.0013

Each month coefficient is the predicted difference in sales between that month and January, holding all five numerical predictors constant.

For example, predicted May sales are approximately 6709.92 cars higher than predicted January sales at the same numerical predictor values. These coefficients are adjusted differences, not simply differences between raw monthly averages.

Only 11 month indicators are included because January is represented by the intercept. Including all 12 indicators along with an intercept would create perfect multicollinearity.

(ii) Training fit and significance

The seasonal model gives

$R^2 = 0.7834,$	$R_{\text{adj}}^2 = 0.7316,$	$\text{RMSE} = 2836.68.$
-----------------	------------------------------	--------------------------

It explains approximately 78.34% of the variation in training sales.
For the five numerical predictors:

Predictor	Estimate	<i>p</i> -value
Unemployment	−1122.0490	0.0123
AccordQueries	272.6950	0.0120
CPIEnergy	−125.7680	0.0004
CPIAll	−27.5306	0.6365
MilesTraveled	0.036875	0.5150

Using individual conventional tests at the 5% level:

- Significant numerical predictors: **Unemployment**, **AccordQueries**, and **CPIEnergy**.
- Significant month indicators relative to January: April, May, November, and December.
- **CPIAll**, **MilesTraveled**, and the other month indicators are not individually significant.

The intercept is not significant at the 5% level ($p = 0.0605$). These tests are unadjusted for multiple comparisons and rely on the conventional OLS inference assumptions described above.

(iii) Does MonthFactor improve the model?

Compared with the model containing all five numerical predictors but no month indicators:

Measure	Without months	With months
Training R^2	0.7092	0.7834
Adjusted R^2	0.6906	0.7316
Training RMSE	3286.47	2836.68
Chronological validation RMSE	15139.30	11156.18

Training R^2 cannot decrease when predictors are added. However, the increase in adjusted R^2 also supports the usefulness of the month indicators.

A partial F -test compares the nested models under

$$H_0 : \delta_2 = \delta_3 = \cdots = \delta_{12} = 0.$$

The reduced model has 6 coefficients, and the seasonal model has 17 coefficients. Therefore,

$$F = \frac{(\text{RSS}_R - \text{RSS}_S)/11}{\text{RSS}_S/(84 - 17)}.$$

Using

$$\text{RSS}_R = 907271716.04, \quad \text{RSS}_S = 675926892.10,$$

gives

$$F(11, 67) = 2.0847, \quad p = 0.0334.$$

Thus, the month indicators are jointly significant at the 5% level under the conventional regression assumptions.

Adding months improves both training fit and chronological validation RMSE relative to the full numerical model. However, the seasonal model's validation RMSE of 11156.18 is still much larger than the selected model's 4732.20.

Therefore, seasonality improves the full model, but this validation analysis does not establish the seasonal model as the best predictive model overall.

(iv) Another way to model seasonality

An alternative is to use the existing `MonthNumeric` variable to construct a sine and cosine pair:

$$S_m = \sin\left(\frac{2\pi m}{12}\right), \quad C_m = \cos\left(\frac{2\pi m}{12}\right),$$

where $m \in \{1, \dots, 12\}$.

For example, an alternative seasonal model could be

$$\widehat{\text{AccordSales}} = \beta_0 + \sum_{j=1}^5 \beta_j X_j + aS_m + bC_m,$$

where $X_1, \dots, X_5$ are the five numerical predictors.

This represents a smooth annual cycle using two seasonal coefficients instead of 11 month-indicator coefficients. It also respects the fact that December and January are adjacent in the annual cycle.

The smaller number of coefficients could reduce estimation variance and improve predictive performance if seasonality is smooth. However, it may miss sharp month-specific changes, such as a December sales increase.

Consequently, it could improve on the month-indicator model, but there is insufficient evidence to expect it to outperform the selected model from part (a) without training-only validation.

This alternative is proposed only for part (b)(iv) and was not added to part (a).

Python Code for Question 2

The following code uses `pandas`, `numpy`, and `scipy`. Place `Accord-242A-Fall126.csv` in the working directory.

The code reproduces the regression coefficients, conventional p -values and confidence intervals, training and validation performance, VIFs, and the partial F -test. No test outcomes are used.

```
import itertools
import numpy as np
import pandas as pd
from scipy.stats import t, f

# -----
# Load data and split chronologically
# -----
df = pd.read_csv("Accord-242A-Fall126.csv")
df["Date"] = pd.to_datetime(df["Date"], errors="raise")
df = df.sort_values("Date").reset_index(drop=True)

train = df.loc[
    df["Date"].between("2016-08-01", "2023-07-31")
].copy()

test = df.loc[
    df["Date"].between("2023-08-01", "2026-07-31")
].copy()

assert len(train) == 84
assert len(test) == 35
assert not df["Date"].duplicated().any()

features = [
    "Unemployment",
    "AccordQueries",
    "CPIEnergy",
    "CPIA11",
    "MilesTraveled",
]

X = train[features].to_numpy(dtype=float)
y = train["AccordSales"].to_numpy(dtype=float)

assert np.isfinite(X).all()
assert np.isfinite(y).all()

print("Training observations:", len(train))
print("Testing observations:", len(test))

# -----
# OLS fitting and conventional inference
# -----
```

```

def fit_ols(X, y):
    X = np.asarray(X, dtype=float)
    y = np.asarray(y, dtype=float)

    A = np.column_stack([np.ones(len(y)), X])
    n, k = A.shape

    assert n > k
    assert np.linalg.matrix_rank(A) == k

    coef = np.linalg.lstsq(A, y, rcond=None)[0]
    fitted = A @ coef
    residuals = y - fitted

    rss = residuals @ residuals
    tss = np.sum((y - y.mean()) ** 2)
    residual_df = n - k

    # Conventional homoskedastic OLS covariance
    covariance = (
        rss / residual_df
        * np.linalg.inv(A.T @ A)
    )

    se = np.sqrt(np.diag(covariance))
    pvalues = 2 * t.sf(
        np.abs(coef / se), residual_df
    )

    critical = t.ppf(0.975, residual_df)
    ci_low = coef - critical * se
    ci_high = coef + critical * se

    return {
        "coef": coef,
        "rss": rss,
        "r2": 1 - rss / tss,
        "adj_r2": 1 - (
            (rss / residual_df) / (tss / (n - 1))
        ),
        "rmse": np.sqrt(rss / n),
        "pvalues": pvalues,
        "ci_low": ci_low,
        "ci_high": ci_high,
        "residual_df": residual_df,
    }

# -----
# Expanding-window chronological validation
# -----
def cv_rmse(X, y):
    errors = []

```

```

    for end in [24, 36, 48, 60, 72]:
        model = fit_ols(X[:end], y[:end])

        X_validation = X[end:end + 12]
        A_validation = np.column_stack([
            np.ones(len(X_validation)),
            X_validation,
        ])

        predicted = A_validation @ model["coef"]
        errors.extend(y[end:end + 12] - predicted)

    return np.sqrt(np.mean(np.asarray(errors) ** 2))

# -----
# Part (a): compare all 31 nonempty subsets
# -----
rows = []

for size in range(1, 6):
    for indices in itertools.combinations(range(5), size):
        subset_X = X[:, indices]

        rows.append({
            "indices": indices,
            "predictors": tuple(
                features[i] for i in indices
            ),
            "count": size,
            "cv_rmse": cv_rmse(subset_X, y),
        })

ranking = pd.DataFrame(rows).sort_values(
    ["cv_rmse", "count"]
).reset_index(drop=True)

print("\nTop subsets")
print(ranking[["predictors", "cv_rmse"]].head(10))

chosen_indices = ranking.loc[0, "indices"]
chosen_names = [features[i] for i in chosen_indices]
chosen_X = X[:, chosen_indices]

selected = fit_ols(chosen_X, y)
full = fit_ols(X, y)

def coefficient_table(model, names):
    return pd.DataFrame({
        "Coefficient": model["coef"],
        "p_value": model["pvalues"],
        "CI_lower": model["ci_low"],
        "CI_upper": model["ci_high"],
    })

```

```

    }, index=["Intercept"] + list(names))

def vif_table(X, names):
    result = []

    for j, name in enumerate(names):
        other_X = np.delete(X, j, axis=1)
        auxiliary = fit_ols(other_X, X[:, j])

        result.append({
            "Predictor": name,
            "VIF": 1 / (1 - auxiliary["r2"]),
        })

    return pd.DataFrame(result)

print("\nSelected model")
print(coefficient_table(selected, chosen_names))

equation = (
    f"Predicted AccordSales = {selected['coef'][0]:.6f}"
)
for name, coefficient in zip(
    chosen_names, selected["coef"][1:]
):
    equation += f" {coefficient:+.6f} * {name}"

print("\nSelected regression equation")
print(equation)

print("\nFull model")
print(coefficient_table(full, features))

print("\nSelected-model VIF")
print(vif_table(chosen_X, chosen_names))

print("\nFull-model VIF")
print(vif_table(X, features))

print("\nPredictor correlations")
print(train[features].corr())

print("\nMean-only training RMSE")
print(np.sqrt(np.mean((y - y.mean()) ** 2)))

# -----
# Part (b): all five predictors plus MonthFactor
# -----
month_names = [
    "January", "February", "March", "April",
    "May", "June", "July", "August",
    "September", "October", "November", "December",

```

```

]

months = train["MonthFactor"].astype(str).str.strip()
assert months.isin(month_names).all()

# January is the omitted reference category.
month_dummies = np.column_stack([
    (months == month).to_numpy(dtype=float)
    for month in month_names[1:]
])

seasonal_X = np.column_stack([X, month_dummies])
seasonal_names = features + month_names[1:]
seasonal = fit_ols(seasonal_X, y)

print("\nSeasonal model")
print(coefficient_table(seasonal, seasonal_names))

print("\nSeasonal coefficients significant at 5%")
seasonal_table = coefficient_table(
    seasonal, seasonal_names
)
print(seasonal_table.loc[
    seasonal_table["p_value"] < 0.05
])

# -----
# Model performance comparison
# -----
comparison = []

for name, model, design_X in [
    ("Selected", selected, chosen_X),
    ("Full", full, X),
    ("Full plus months", seasonal, seasonal_X),
]:
    comparison.append({
        "Model": name,
        "Training_R2": model["r2"],
        "Adjusted_R2": model["adj_r2"],
        "Training_RMSE": model["rmse"],
        "Validation_RMSE": cv_rmse(design_X, y),
        "RSS": model["rss"],
    })

print("\nModel comparison")
print(pd.DataFrame(comparison).to_string(index=False))

# -----
# Partial F-test for the 11 month indicators
# -----
q = 11

F_statistic = (

```

```
    (full["rss"] - seasonal["rss"]) / q
) / (
    seasonal["rss"] / seasonal["residual_df"]
)

F_pvalue = f.sf(
    F_statistic, q, seasonal["residual_df"]
)

print("\nJoint test of month indicators")
print("F statistic:", F_statistic)
print("Numerator df:", q)
print("Denominator df:", seasonal["residual_df"])
print("p-value:", F_pvalue)
```